

AXIOM Semantic Cache

System Documentation

Version: 1.0 (Stable) · **Project:** AXIOM V2.0 AI Services **Purpose:** This document explains how the AXIOM Semantic Cache works, how to deploy it, and what data can be used to seed it. It is written so that anyone — technical or not — can understand the system.

Version 2.0 (planned) will introduce database scalability controls. See [Section 15](#) for the full design.

Table of Contents

1. [What Is a Semantic Cache?](#)
 2. [Why Does It Matter?](#)
 3. [System Architecture](#)
 4. [How a Query Flows Through the System](#)
 5. [The Three-Tier Cache](#)
 6. [The Privacy Gatekeeper — Query Classifier](#)
 7. [The Embedding Model](#)
 8. [The Database Layer — Supabase](#)
 9. [The Test Console \(Frontend\)](#)
 10. [Seeding the Cache — Datasets](#)
 11. [Running the System Locally](#)
 12. [Deployment Options](#)
 13. [API Reference](#)
 14. [Glossary](#)
 15. [Version 2 Roadmap — Scalability](#)
-

1. What Is a Semantic Cache?

A **regular cache** remembers exact answers to exact questions. If someone asks *"What is Python?"* and you already answered it, you return the saved answer instead of calling an AI model again.

A **semantic cache** is smarter. It understands the *meaning* of a question, not just the exact words. So if someone asks:

- "What is Python?"
- "Can you explain what Python is?"
- "Tell me about the Python programming language"

...the system recognizes all three as the **same question** and returns the cached answer for all of them — even though the words are completely different.

This is the core idea behind AXIOM's Semantic Cache.

2. Why Does It Matter?

Every time an AI model (like GPT-4, Claude, or Groq) generates a response, it costs:

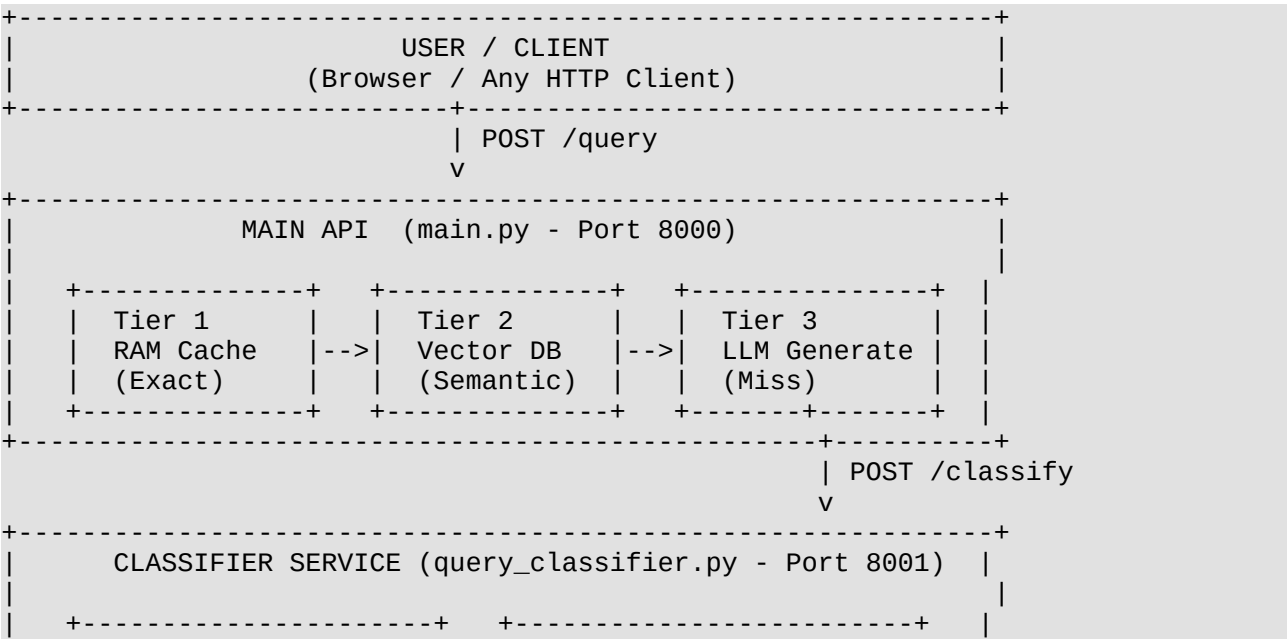
Cost	Impact
Money	API calls are billed per token
Time	Generation takes 1-10 seconds
Energy	GPU compute is energy-intensive

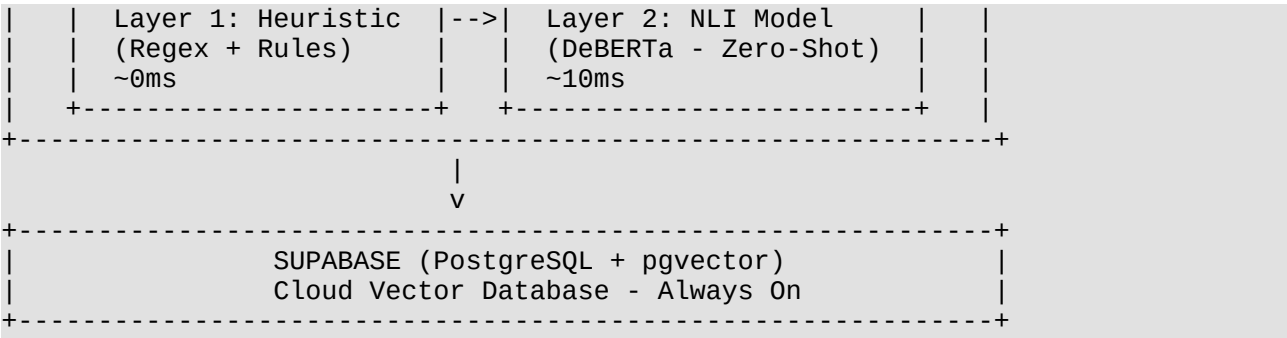
If thousands of users ask the same general questions (e.g., "*How does photosynthesis work?*"), generating a fresh response every time is wasteful. A semantic cache **answers instantly** from stored responses, saving all three resources.

Key insight: The cache only stores answers to *general* questions. Personal queries (like "*What is my account balance?*") are never cached — the system automatically detects and skips them.

3. System Architecture

The system consists of **four main components** that work together:



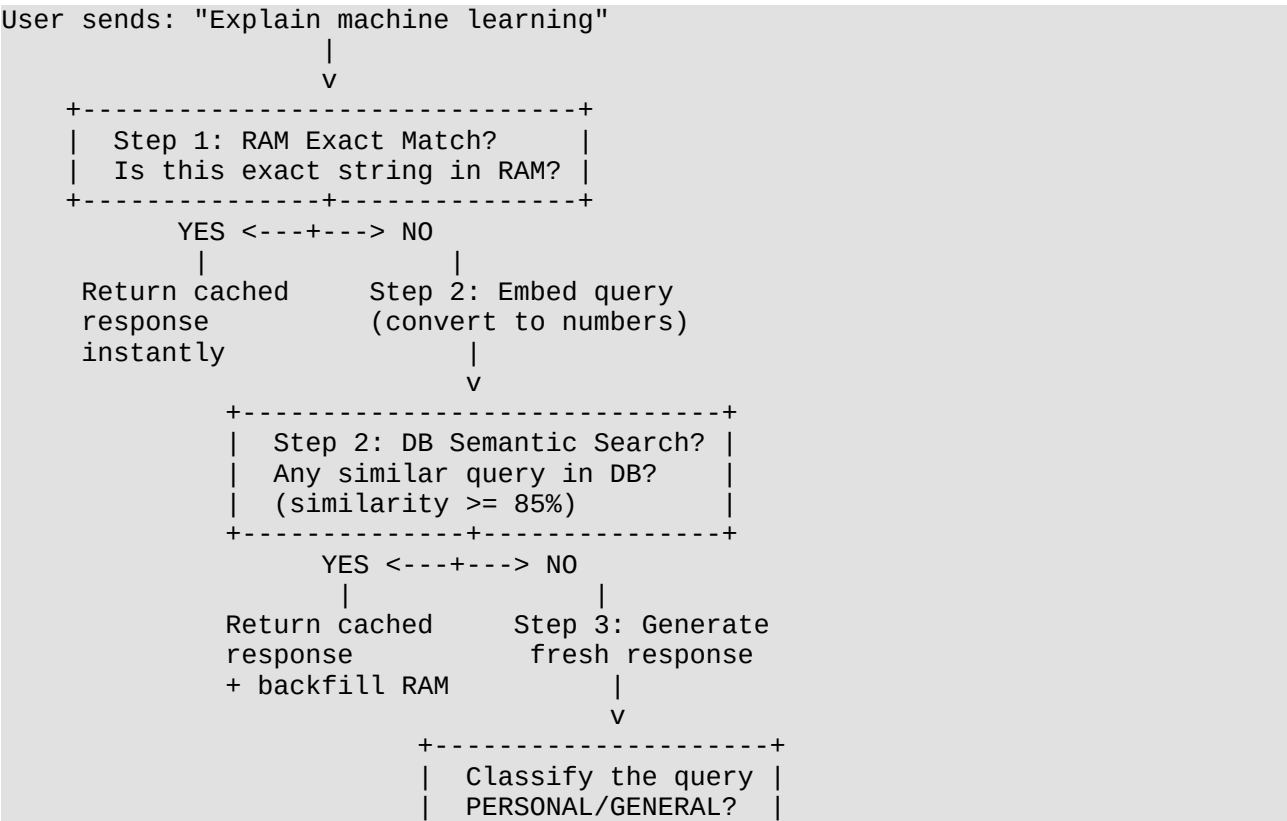


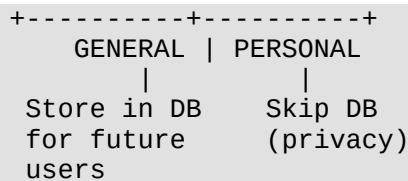
Components at a glance:

Component	File	Port	Role
Main Cache API	main.py	8000	Receives queries, manages all 3 tiers
Query Classifier	query_classifier.py	8001	Decides if a query is personal or general
Vector Database	Supabase (cloud)	—	Stores embeddings + cached responses
Test Console	frontend/index.html	—	Browser UI to interact with the system

4. How a Query Flows Through the System

Here is the step-by-step journey of any query:





5. The Three-Tier Cache

The cache has three layers, each faster than the next:

Tier 1 — RAM Exact Match

- **Speed:** Less than 1 millisecond
- **How it works:** A Python dictionary in memory. If the exact query string was seen before in this session, the answer is returned instantly.
- **Limit:** Holds up to 10,000 queries. When full, the oldest entry is removed to make room.
- **Scope:** Per-session only. Resets when the server restarts.

Tier 2 — Database Semantic Match

- **Speed:** 50–200 milliseconds
- **How it works:** The query is converted into a mathematical vector (768 numbers) using an embedding model. That vector is compared against all stored vectors in the database using cosine similarity. If a match is found with 85% or higher similarity, the stored answer is returned.
- **Scope:** Persistent. Shared across all users, survives restarts.
- **Threshold:** 0.85 similarity score (tunable in `main.py`)

Tier 3 — LLM Generation

- **Speed:** 1,000–10,000 milliseconds (depends on LLM provider)
- **How it works:** No cached answer was found. A real AI model generates a fresh response.
- **What happens next:** The response is stored in RAM immediately. If the query is classified as GENERAL, it is also stored in the database for future users.

6. The Privacy Gatekeeper — Query Classifier

The classifier is the system's privacy guard. It runs **only on cache misses** (when the system needs to decide whether to store a new response). It never runs on cache hits, so it adds zero cost to fast queries.

Why Is This Needed?

Imagine a user asks: "What is my bank account balance?"

If this were stored in the shared cache, the next user who asked a similar question might receive someone else's private financial data. The classifier prevents this.

Two Classification Layers

Layer 1 — Heuristic (Rule-Based) — approximately 0ms

A set of hand-crafted rules that catch obvious personal queries instantly:

- Detects PII patterns: email addresses, phone numbers, credit card numbers, SSNs
- Detects personal action phrases: "remind me", "cancel my", "what is my balance", "show me my orders"
- Detects possessive + personal noun combos: "my account", "our records", "my prescription"

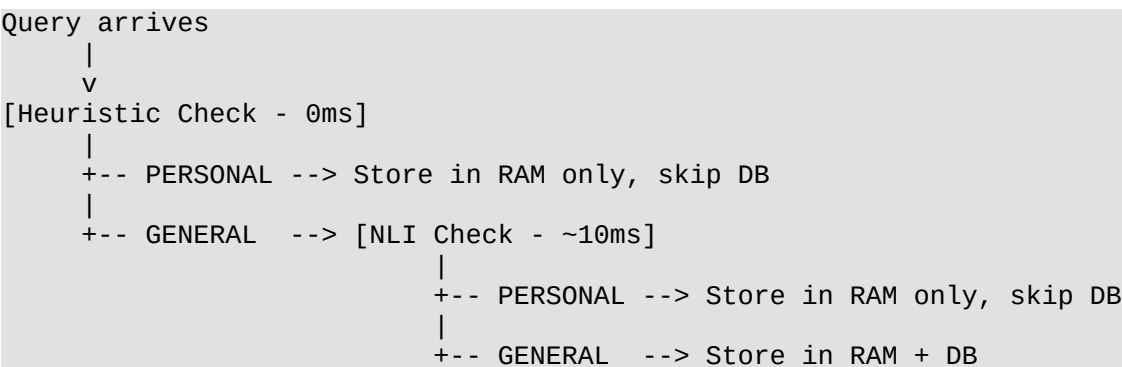
If heuristics flag the query as PERSONAL, the decision is final and NLI is skipped.

Layer 2 — NLI Model (AI-Based) — approximately 10ms

If heuristics pass the query as GENERAL, the NLI model does a deeper check.

- **Model:** cross-encoder/nli-deberta-v3-small
- **Technique:** Zero-shot classification — no task-specific training needed
- **Question asked internally:** "Is this question about the speaker's own private account data, or about general world knowledge?"
- **Threshold:** If the model is 60% or more confident the query is personal, it is classified as PERSONAL

Classification Decision Flow



Example Classifications

Query	Layer	Decision	Reason
"What is my account balance?"	Heuristic	PERSONAL	Possessive + personal noun
"Remind me to call the	Heuristic	PERSONAL	Personal action phrase

Query	Layer	Decision	Reason
doctor"			
"john@gmail.com password reset"	Heuristic	PERSONAL	PII pattern detected
"How does photosynthesis work?"	NLI	GENERAL	About world knowledge
"What causes inflation?"	NLI	GENERAL	About world knowledge
"Cancel my subscription"	Heuristic	PERSONAL	Personal action phrase

7. The Embedding Model

Model: all-mpnet-base-v2 (by Sentence Transformers)

An embedding model converts text into a list of numbers (a "vector") that captures its meaning. Two sentences with similar meanings will produce vectors that are mathematically close to each other — even if the words are completely different.

Property	Value
Output dimensions	768
Max input length	514 tokens
Model size	~420 MB
GPU support	Yes (CUDA)
Why this model	Best balance of accuracy and speed for semantic similarity

How similarity is measured:

Cosine similarity compares the "angle" between two vectors. A score of:

- 1.00 = identical meaning
- 0.85+ = very similar (cache hit threshold)
- 0.50 = loosely related
- 0.00 = completely unrelated

8. The Database Layer — Supabase

Supabase is a cloud database built on PostgreSQL. The system uses its `pgvector` extension to store and search vector embeddings at scale.

Table Structure: `shared_llm_cache`

Column	Type	Description
<code>id</code>	UUID	Unique row identifier
<code>query_text</code>	TEXT	The original question text
<code>response_text</code>	TEXT	The cached AI response
<code>embedding</code>	VECTOR(768)	Mathematical representation of the query
<code>created_at</code>	TIMESTAMP	When this entry was added

How Vector Search Works

The database function `match_shared_cache` performs an approximate nearest-neighbor search using the query embedding. It returns the closest matching entry above the similarity threshold in milliseconds, even with millions of rows.

9. The Test Console (Frontend)

The system includes a browser-based test console (`frontend/index.html`) for testing and demonstration.

Features:

- Send queries directly to the cache API
- See color-coded responses by source (RAM / DB / LLM)
- View live session statistics (total queries, RAM hits, DB hits, LLM misses)
- Real-time cache hit rate bar
- API online/offline status indicator
- Quick prompt buttons for common test queries

How to use: Open `frontend/index.html` in a browser while the API is running. No server needed — it's a static HTML file that calls the API directly.

10. Seeding the Cache — Datasets

"Seeding" means pre-loading the database with high-quality question-answer pairs before any real users arrive. This is what makes the cache useful from day one.

How the Seeding Script Works (`seed_cache.py`)

1. **Streams** the dataset file item-by-item using `json` (no full file loaded into RAM — avoids memory crashes on large files)

2. **Filters** out queries longer than 500 characters
3. **Deduplicates** by checking against existing entries in the database
4. **Accumulates** valid records into a batch of 2,000
5. **Batch-embeds** the entire batch at once on GPU (much faster than one-by-one)
6. **Batch-inserts** all 2,000 records to Supabase in a single API call
7. **Saves a checkpoint** after each batch (resumable if interrupted)
8. **Repeats** until the target count is reached

Configuration

Setting	Default	Description
SEED_COUNT	8,000	Total rows to insert this run
BATCH_SIZE	2,000	Records per embedding + insert cycle
MAX_QUERY_CHARS	500	Skip queries longer than this

Recommended Datasets

The following datasets are well-suited for seeding the AXIOM cache. They contain general-knowledge Q&A pairs that are non-personal and high quality.

OpenHermes 2.5 — Primary Dataset

- **Source:** [teknium/OpenHermes-2.5](#) on Hugging Face
 - **Size:** ~1 million conversations
 - **Format:** Multi-turn chat (human/assistant pairs)
 - **Content:** Coding, reasoning, science, general knowledge, creative writing
 - **Why it's great:** Extremely diverse topics, high quality, instruction-following style that maps well to real-world queries
 - **Target rows:** 8,000–15,000
 - **Status:** Currently in use
-

ShareGPT (Cleaned)

- **Source:** [anon8231489123/ShareGPT_Vicuna_unfiltered](#) on Hugging Face
- **Size:** ~90,000 conversations
- **Content:** Real user conversations with ChatGPT — covers an enormous range of everyday topics

- **Why it's great:** Represents real-world query diversity better than curated datasets; conversations feel natural
 - **Caution:** Contains some personal-sounding queries — pre-filter before seeding if possible
 - **Target rows:** 3,000–5,000
-

Dolly 15K (by Databricks)

- **Source:** databricks/databricks-dolly-15k on Hugging Face
 - **Size:** 15,000 examples
 - **Format:** Instruction + response (single turn)
 - **Content:** Open QA, classification, summarization, general knowledge
 - **Why it's great:** Curated by human annotators — very clean, accurate responses; great for factual Q&A
 - **Target rows:** 5,000–10,000 (can use the full dataset)
-

FLAN Collection (Google)

- **Source:** Muennighoff/flan on Hugging Face
 - **Size:** Millions of examples
 - **Content:** Task-oriented reasoning, science, math, general knowledge
 - **Why it's great:** Excellent coverage of factual questions; used to train many open-source models
 - **Caution:** Very large — use a filtered subset of ~10,000 rows
-

WizardLM Evol-Instruct

- **Source:** WizardLM/WizardLM_evol_instruct_70k on Hugging Face
 - **Size:** 70,000 examples
 - **Content:** Step-by-step reasoning, coding, problem solving
 - **Why it's great:** Good for users who ask complex or multi-step questions
 - **Target rows:** 3,000–5,000
-

Recommended Seeding Strategy

For a hackathon or MVP deployment:

Phase 1 – OpenHermes 2.5:	8,000 rows	(broad coverage)
Phase 2 – Dolly 15K:	5,000 rows	(factual, clean)

```
Phase 3 – ShareGPT Cleaned: 3,000 rows (real-world diversity)
-----
Total: 16,000 rows
```

16,000 high-quality rows provides strong cache hit rates for general Q&A use cases while keeping the database lean and fast.

11. Running the System Locally

Prerequisites

- Python 3.10+
- A Supabase project with the `shared_llm_cache` table and `pgvector` enabled
- NVIDIA GPU (optional, but recommended for faster embedding)

Installation

```
cd "AXIOM V2.0/AI_SERVICES/semantic_cache"

python -m venv .venv
.venv\Scripts\activate      # Windows
source .venv/bin/activate   # Linux/macOS

pip install -r requirements.txt
```

Configuration

Edit `.env` with your Supabase credentials:

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your-anon-key
HF_TOKEN=your-huggingface-token
```

Starting the Services

Open two terminal windows and run one command in each:

Terminal 1 — Main Cache API (Port 8000):

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Terminal 2 — Classifier Service (Port 8001):

```
uvicorn query_classifier:app --host 0.0.0.0 --port 8001 --reload
```

Seeding the Database

```
python seed_cache.py
```

Expected output per batch cycle:

```
[BATCH] Embedding 2000 queries on CUDA...
[BATCH] Inserting 2000 records to Supabase...
[BATCH] Inserted 2000 records. Total so far: 2000/8000
```

[CHECKPOINT] Mid-run checkpoint saved

Testing

Open `frontend/index.html` in your browser, or send a direct request:

```
curl -X POST http://localhost:8000/query \  
-H "Content-Type: application/json" \  
-d '{"prompt": "What is machine learning?"}'
```

12. Deployment Options

Option A — Self-Host + Cloudflare Tunnel (Recommended for Hackathons)

Run the API on your own machine and expose it publicly using Cloudflare Tunnel — completely free.

Pros: Full GPU access, no RAM limits, no cold starts, zero cost **Cons:** PC must stay on; not suitable for 24/7 production

Steps:

1. Install `cloudflared` from cloudflare.com
2. Start your API: `uvicorn main:app --host 0.0.0.0 --port 8000`
3. Expose it: `cloudflared tunnel --url http://localhost:8000`
4. You receive a permanent public HTTPS URL

Option B — Oracle Cloud Always Free (Best Free Cloud Option)

Oracle Cloud's Always Free tier provides a real cloud server at no cost, forever.

Specs available for free:

- Up to 4 ARM vCPUs
- Up to 24 GB RAM
- Always on — no sleep, no expiry

Why it works: 24 GB RAM easily handles both the embedding model (420 MB) and the NLI model (500 MB) with plenty of headroom.

Steps:

1. Create a free Oracle Cloud account at cloud.oracle.com
2. Create an Ampere A1 instance (Ubuntu 22.04)
3. SSH in, install Python, clone the project

4. Run services with `systemd` for automatic restart on reboot

5. Open port 8000 in Oracle's firewall rules

Tip: Ampere A1 capacity can be limited in popular regions. Try Frankfurt or Osaka if your home region shows no availability.

Option C — Render / Railway (Not Recommended)

These platforms have two critical limitations:

- Services **sleep after 15 minutes** of inactivity — cold start takes 30–60 seconds
- Free RAM is **512 MB** — not enough to load both ML models

Not suitable for this system without major model optimization.

Deployment Comparison

Option	Cost	Always On	RAM	GPU	Best For
Self-Host + Cloudflare Tunnel	Free	Yes (while PC is on)	Your PC's RAM	Your GPU	Hackathons, demos
Oracle Cloud Always Free	Free	Yes	24 GB	CPU only	Long-term, production
Render / Railway	Free	No (sleeps)	512 MB	No	Not recommended
Hugging Face Spaces (Docker)	Paid	Yes	Configurable	Optional	Paid production

13. API Reference

Main Cache API — Port 8000

POST /query

The primary endpoint. Sends a query through the full three-tier cache pipeline.

Request:

```
{
  "prompt": "What is machine learning?"
}
```

Response — RAM Cache Hit:

```
{
```

```
{
  "status": "success",
  "source": "RAM_Exact_Hit",
  "response": "Machine learning is a subset of AI...",
  "debug": {
    "tier": "RAM",
    "cached": true,
    "classifier_called": false,
    "classifier_note": "Classifier not called on cache hits"
  }
}
```

Response — DB Semantic Hit:

```
{
  "status": "success",
  "source": "DB_Semantic_Hit",
  "response": "Machine learning is a subset of AI...",
  "debug": {
    "tier": "DB",
    "cached": true,
    "similarity_score": 0.9124,
    "classifier_called": false
  }
}
```

Response — Cache Miss (LLM Generated):

```
{
  "status": "success",
  "source": "LLM_Generation_Miss",
  "response": "Machine learning is...",
  "debug": {
    "tier": "LLM",
    "cached": false,
    "classifier_called": true,
    "query_type": "GENERAL",
    "decision_layer": "NLI",
    "heuristic_reason": null,
    "db_stored": true
  }
}
```

Classifier Service — Port 8001

POST /classify

Full two-layer classification (recommended for production).

Request:

```
{ "query": "What is my account balance?" }
```

Response:

```
{
  "query": "What is my account balance?",
  "final_label": "PERSONAL",
  "decision_layer": "HEURISTIC",
  "heuristic_label": "PERSONAL",
  "heuristic_reason": "Possessive + personal noun: 'my account'",
}
```

```
"nli_label": null,  
"nli_confidence": null,  
"nli_scores": null,  
"total_latency_ms": 0.42  
}
```

POST /classify/heuristic

Heuristic layer only. Instant, rule-based result.

POST /classify/nli

NLI model only. AI-powered zero-shot classification.

POST /classify/compare

Runs both layers independently and returns results side-by-side. Useful for testing and calibrating the system.

GET /health

Returns service status, model name, and current NLI threshold.

14. Glossary

Term	Meaning
Semantic Cache	A cache that matches queries by meaning, not just exact text
Embedding	Converting text into a list of numbers that represents its meaning
Vector	A list of numbers representing a piece of text in mathematical space
Cosine Similarity	A measure of how similar two vectors are (0 = unrelated, 1 = identical)
pgvector	A PostgreSQL extension that enables fast vector similarity search
NLI	Natural Language Inference — an AI technique for reasoning about text relationships
Zero-shot Classification	Classifying text into categories without task-specific training
DeBERTa	A transformer model from Microsoft used for NLI tasks
Heuristic	A rule-based approach that uses patterns and logic instead of AI
Cold Start	The delay that occurs when a service wakes up from sleep and reloads its models
Backfill	When a DB cache hit is also stored in RAM for

Term	Meaning
	even faster access next time
Checkpoint	A saved state that allows a long-running process to resume if interrupted
Supabase	A cloud database platform built on PostgreSQL
Cloudflare Tunnel	A free tool that creates a secure public URL pointing to your local machine
SEED_COUNT	The target number of rows to insert during a seeding run
BATCH_SIZE	How many records are embedded and inserted at once for efficiency

15. Version 2 Roadmap — Scalability

Status: Planned for Version 2.0. No code changes have been made in V1. This section documents the agreed design so the team can implement it when ready.

The Problem

The current system is a **self-growing database** — every new general query that results in a cache miss gets stored permanently. Over time this creates two problems:

1. **Unbounded growth** — the database will grow indefinitely with no upper limit
2. **Stale data** — old, rarely-used entries accumulate and slow down vector search

The Solution — LFU Eviction with GitHub Actions

Version 2 will introduce a **Least Frequently Used (LFU)** eviction policy that automatically removes low-value entries when the database approaches its capacity limit. The eviction runs on a scheduled cron job hosted on **GitHub Actions** — meaning it operates entirely in the cloud, independent of any local machine or deployed server.

Capacity Design Parameters

Parameter	Value	Meaning
Max DB rows	50,000	Hard ceiling (safe for Supabase free tier ~500MB)
Eviction threshold	80% = 40,000 rows	Eviction triggers when DB crosses this
Eviction target	70% = 35,000 rows	DB is pruned back to this level
Grace period	7 days	Newly inserted rows cannot be evicted

Parameter	Value	Meaning
Recency window	30 days	"Recent" access counts double in scoring
Schedule	Every 2 days	GitHub Actions cron

Storage estimate per row: ~3.7 KB (768-dim float32 vector + query + response text)

Rows	Estimated Storage
10,000	~37 MB
50,000	~185 MB
75,000	~278 MB
100,000	~370 MB

LFU Scoring Formula

Every row gets a score. Lowest scorers are deleted first.

```
score = lifetime_hit_count
      + lifetime_hit_count  (×2 bonus if accessed within last 30 days)
      + IMMUNE              (if row is less than 7 days old)
```

Rows with the lowest score are evicted first.
Rows within the grace period are never touched.

This formula favours:

- **Frequently used** entries (high hit count)
- **Recently active** entries (double weight in recency window)
- **New entries** are protected so they get a fair chance to accumulate hits before being judged

Why GitHub Actions as the Scheduler

GitHub Actions was chosen over alternatives because:

Option	Why Rejected
Local cron / Task Scheduler	PC must be on — not reliable
Supabase pg_cron	Requires Supabase Pro (\$25/mo)
Render / Railway scheduled job	Platform dependency, potential costs
GitHub Actions	Free, cloud-hosted, no dependencies, manual trigger available

The GitHub Actions workflow makes a single HTTP call to a Supabase stored procedure

(`evict_lfu_cache`). The entire eviction logic lives inside the database — the workflow just triggers it. This means:

- No Python runtime needed in the action
- The eviction function is reusable from any trigger source
- Execution takes ~3–5 seconds
- Free tier gives 2,000 minutes/month (this uses ~0.1 minutes per run)

Database Schema Changes Required (V2)

Two new columns will be added to `shared_llm_cache`:

Column	Type	Default	Purpose
<code>hit_count</code>	INTEGER	0	Incremented every time this row is served as a cache hit
<code>last_accessed_at</code>	TIMESTAMPTZ	<code>now()</code>	Updated on every cache hit

SQL to run in Supabase SQL Editor:

```
ALTER TABLE shared_llm_cache
  ADD COLUMN IF NOT EXISTS hit_count          INTEGER          NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS last_accessed_at    TIMESTAMPTZ      NOT NULL DEFAULT now();

CREATE INDEX IF NOT EXISTS idx_cache_hit_count
  ON shared_llm_cache (hit_count ASC, last_accessed_at ASC);
```

Code Changes Required (V2)

main.py — On every DB semantic hit, fire an async update:

```
# After a DB hit is served, increment the hit counter for that row
supabase.table("shared_llm_cache").update({
  "hit_count": matched_row["hit_count"] + 1,
  "last_accessed_at": "now()"
}).eq("id", matched_row["id"]).execute()
```

.github/workflows/evict_cache.yml — New workflow file:

```
name: LFU Cache Eviction

on:
  schedule:
    - cron: '0 2 */2 * *' # Every 2 days at 2:00 AM UTC
  workflow_dispatch:      # Manual trigger from GitHub UI

jobs:
  evict:
    runs-on: ubuntu-latest
```

```
steps:
  - name: Trigger evict_lfu_cache via Supabase RPC
    run: |
      curl -s -X POST "${{ secrets.SUPABASE_URL
}}/rest/v1/rpc/evict_lfu_cache" \
      -H "apikey: ${{ secrets.SUPABASE_SERVICE_KEY }}" \
      -H "Authorization: Bearer ${{ secrets.SUPABASE_SERVICE_KEY }}" \
      -H "Content-Type: application/json" \
      -d '{"p_max_rows": 50000, "p_threshold_pct": 0.80, "p_target_pct":
0.70}'
```

GitHub Secrets needed:

- SUPABASE_URL — your Supabase project URL
- SUPABASE_SERVICE_KEY — the service_role key (not anon key)

Version Comparison

Feature	Version 1.0	Version 2.0
Three-tier cache	Yes	Yes
Privacy classifier	Yes	Yes
Seeded dataset	Yes	Yes
DB eviction	No	Yes (LFU)
DB size cap	No	50,000 rows
Hit tracking	No	Yes
Scheduled maintenance	No	Yes (GitHub Actions)
GitHub repo	No	Yes

Documentation generated for AXIOM V2.0 — AI Services Semantic Cache System — July 2026 V1.0 Stable · V2.0 Scalability Roadmap documented